



WinItalPascal – Msg

Libreria di utilità per applicazioni VB.NET WinForms

Autore: ItalPascal

Data: 19/09/2026

Informazioni sul progetto

Snodo: Msg

WinItalPascal – Libreria VB.NET WinForms

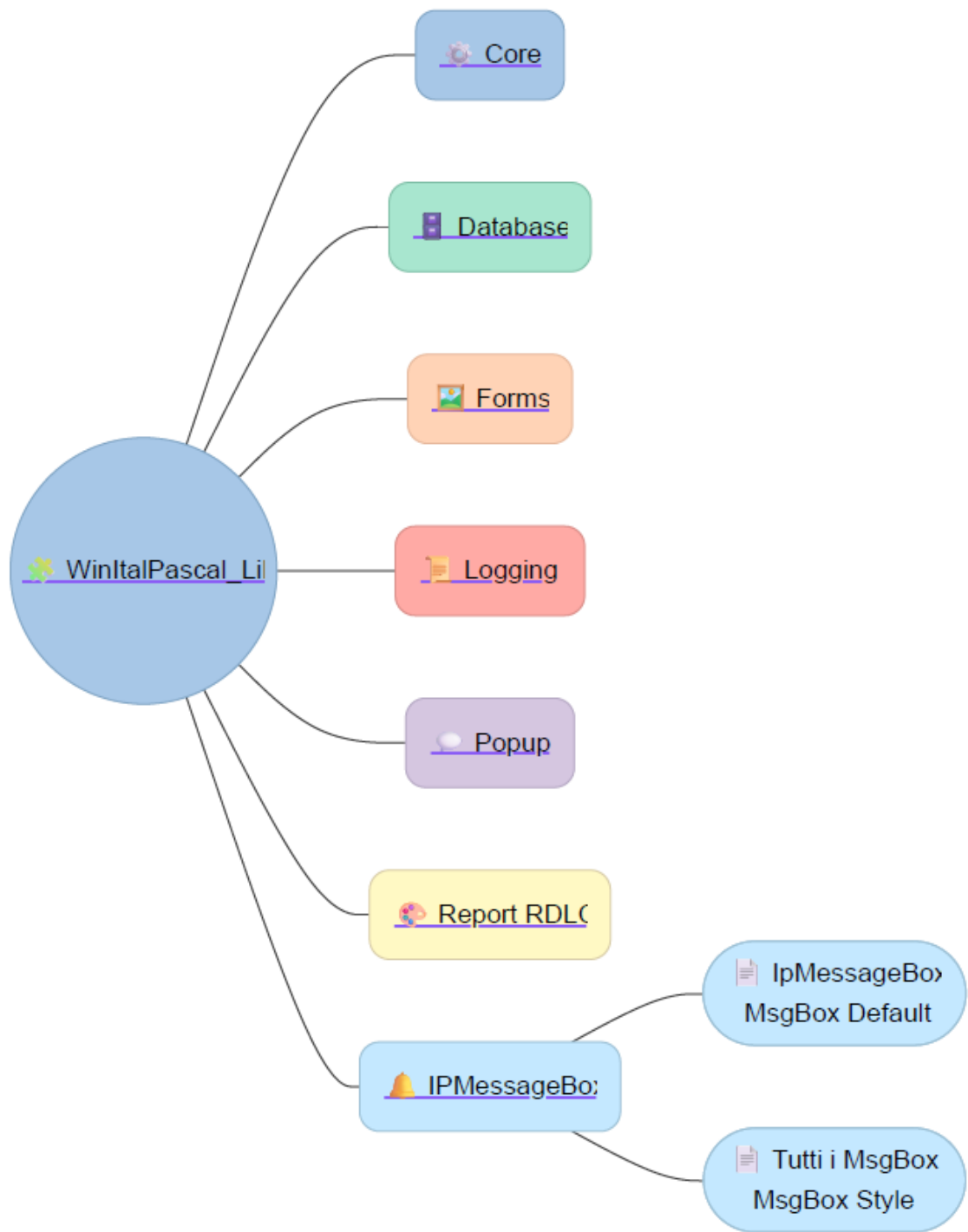
Autore: ItalPascal

Repository GitHub: https://github.com/List051/WinItalPascal_Lib

NuGet: <https://www.nuget.org/packages/WinItalPascal>

Documentazione: <https://list051.github.io/WinVideoShowcase/>

Struttura dello Snodo: Msg



Indice

1. Esempi_IPMessageBox.pdf
2. IPMessageBox.pdf

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE



1. Messaggio semplice (solo testo)

```
1 IPMessageBox.Show("Operazione completata.")
```

Equivalente al classico:

```
1 MessageBox.Show("Operazione completata.")
```



2. Messaggio con titolo

```
1 IPMessageBox.Show("Salvataggio completato correttamente.",  
"Informazione")
```



3. Messaggio con pulsanti OK / Cancel

```
1 Dim result As DialogResult =  
2     IPMessageBox.Show("Vuoi procedere con l'operazione?", "Conferma",  
3     MessageBoxButtons.OKCancel)  
4  
5 If result = DialogResult.OK Then  
6     ' Azione confermata  
7 Else  
8     ' Azione annullata  
9 End If
```

★ 4. Messaggio con icona (Information, Error, Warning, Question)

```
1 IPMessageBox.Show(  
2     "Connessione stabilita correttamente.",  
3     "Informazione",  
4     MessageBoxButtons.OK,  
5     MessageBoxIcon.Information  
6 )
```

★ 5. Messaggio con icona + pulsanti + default button

```
1 Dim result As DialogResult =  
2     IPMessageBox.Show(  
3         "Vuoi eliminare definitivamente il file?",  
4         "Attenzione",  
5         MessageBoxButtons.YesNoCancel,  
6         MessageBoxIcon.Exclamation,  
7         MessageBoxDefaultButton.Button2  
8     )  
9  
10 Select Case result  
11     Case DialogResult.Yes  
12         ' Elimina  
13     Case DialogResult.No  
14         ' Non eliminare  
15     Case DialogResult.Cancel  
16         ' Annulla  
17 End Select
```

★ 6. Messaggio stile “Retry / Cancel”

```
1 Dim result As DialogResult =  
2     IPMessageBox.Show(  
3         "Impossibile completare l'operazione.",  
4         "Errore",  
5         MessageBoxButtons.RetryCancel,  
6         MessageBoxIcon.Error  
7     )  
8  
9 If result = DialogResult.Retry Then  
10     ' Riprova
```

```
11 Else
12     ' Annulla
13 End If
```

★ 7. Messaggio con icona personalizzata (None → chat)

```
1 IPMessageBox.Show(
2     "Questa è una notifica generica.",
3     "Messaggio",
4     MessageBoxButtons.OK,
5     MessageBoxIcon.None
6 )
```

★ 8. Messaggio con colore primario personalizzato

```
1 Dim msg As New IPMessageBox("Operazione completata!", "Successo")
2 msg.PrimaryColor = Color.MediumSeaGreen
3 msg.ShowDialog()
```

★ 9. Messaggio con tre pulsanti (Abort / Retry / Ignore)

```
1 Dim result As DialogResult =
2     IPMessageBox.Show(
3         "Errore durante la lettura del file.",
4         "Errore",
5         MessageBoxButtons.AbortRetryIgnore,
6         MessageBoxIcon.Error
7     )
8
9 Select Case result
10     Case DialogResult.Abort
11         ' Chiudi
12     Case DialogResult.Retry
13         ' Riprova
14     Case DialogResult.Ignore
15         ' Ignora
16 End Select
```

★ 10. Messaggio centrato con testo lungo (auto-resize)

```
1 IPMessageBox.Show(  
2     "Questo è un messaggio molto lungo che verrà automaticamente  
   adattato " &  
3     "alla dimensione della finestra grazie alla proprietà MaximumSize.",  
4     "Informazione"  
5 )
```

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE

◆ Scopo complessivo

Il file definisce la classe `IPMessageBox`, un *wrapper* statico che fornisce un'interfaccia simile a `MessageBox.Show` ma con la possibilità di utilizzare un form personalizzato (`FormMessageBox`).

L'obiettivo è offrire una serie di overloads di `Show` che accettano gli stessi parametri di `MessageBox.Show`, ma che internamente creano e mostrano un'istanza di `FormMessageBox`, garantendo al contempo il corretto rilascio delle risorse tramite l'istruzione `Using`.

Componenti chiave e funzioni

Elemento	Descrizione
<code>Public NotInheritable Class IPMessageBox</code>	Classe non ereditabile (equivalente a <code>static</code> in C#).
<code>Private Sub New()</code>	Costruttore privato per impedire l'istanza della classe.
<code>Public Shared Function Show(...) As DialogResult</code>	Sei overloads di <code>Show</code> (3 senza proprietario, 3 con proprietario). Ogni overload accetta parametri diversi (testo, caption, pulsanti, icona, pulsante predefinito) e restituisce un <code>DialogResult</code> .
<code>Using msgForm = New FormMessageBox(...)</code>	Creazione di un'istanza di <code>FormMessageBox</code> con i parametri ricevuti. L'istruzione <code>Using</code> garantisce che l'oggetto venga automaticamente <code>Dispose</code> al termine del blocco.
<code>msgForm.ShowDialog()</code> / <code>msgForm.ShowDialog(owner)</code>	Mostra il form in modalità modale e restituisce il risultato dell'azione dell'utente.

Flusso di esecuzione

1. Chiamata a `IPMessageBox.Show(...)`

L'applicazione chiama uno degli overloads, passando i parametri desiderati.

2. Creazione del form

All'interno del blocco `Using`, viene istanziato `FormMessageBox` con i parametri ricevuti.

3. Visualizzazione modale

- Se non è stato specificato un proprietario, viene eseguita `msgForm.ShowDialog()`.
- Se è stato passato un `owner`, viene eseguita `msgForm.ShowDialog(owner)`.

4. Raccolta del risultato

Il valore restituito da `ShowDialog` (di tipo `DialogResult`) viene memorizzato nella variabile `result`.

5. Disposizione del form

Al termine del blocco `Using`, `msgForm.Dispose()` viene chiamato automaticamente.

6. Restituzione

L'overload restituisce il valore di `result` all'applicazione chiamante.

◆ Conclusione

`IPMessageBox` è un semplice helper che consente di mostrare un form di messaggio personalizzato con un'interfaccia familiare a `MessageBox`.

La struttura è chiara: un costruttore privato, overload di `Show` che creano un `FormMessageBox`, lo mostrano modally e restituiscono il risultato.

Con qualche piccola refactoring (parametri opzionali, metodo interno comune) il codice diventerebbe più conciso e manutenibile.